Vite & Gourmand

Gestion de projet

Méthode, organisation, planification et outils

ECF Développeur Web et Web Mobile — FastDev — Novembre 2026

1. Contexte et organisation

Le projet est mené pour le compte de **Vite & Gourmand** (Julie et José), au sein de l'agence **FastDev**, sous la direction d'un chef de projet qui a recueilli les besoins. Le développement est réalisé par un développeur web full-stack, chargé de l'ensemble de la chaîne : analyse, maquettage, intégration, développement, tests et déploiement.

Rôle	Personne	Responsabilités
Clients / utilisateurs métier	Julie et José	Expriment le besoin, valident les fonctionnalités
Chef de projet	FastDev	Recueil des besoins, priorisation, suivi des livraisons
Développeur full-stack	Moi	Conception, développement, tests, documentation, déploiement

Livrables

- Dépôt GitHub public : github.com/Lakagoull/vite-et-gourmand (code, scripts SQL, README) ;
- Application déployée : vite-et-gourmand-grii.onrender.com ;
- Documents PDF : charte graphique et maquettes, manuel d'utilisation, documentation technique, présent document ;
- Outil de gestion de projet : tableau Kanban (voir partie 3).

2. Méthode de travail

2.1 Une démarche agile en Kanban

J'ai retenu une **méthode agile de type Kanban**, adaptée à un développeur seul : pas de cérémonies lourdes, mais un flux de travail visible, un nombre limité de tâches en cours et des livraisons fréquentes.

- **Découpage en fonctionnalités** : le cahier des charges a été transformé en *user stories* regroupées en lots cohérents, chacun livrable et testable seul (une branche Git par lot).
- **Limitation du travail en cours** : une seule fonctionnalité en développement à la fois, terminée et testée avant de passer à la suivante.
- **Définition de « terminé »** : une carte n'est terminée que si le code fonctionne, que les cas d'erreur et la sécurité ont été testés, que l'affichage a été vérifié (bureau et mobile) et que la branche est fusionnée dans `develop`.
- **Amélioration continue** : les difficultés rencontrées ont été notées et corrigées au fil de l'eau (voir partie 6).

2.2 Priorisation (MoSCoW)

Priorité	Fonctionnalités
Must indispensable	Base de données, catalogue et filtres, détail du menu, comptes et rôles, commande avec calcul du prix, espaces utilisateur / employé / administrateur, statistiques MongoDB, sécurité, déploiement
Should important	E-mails automatiques, suivi des commandes, avis clients, gestion des horaires, accessibilité RGAA, pages légales
Could confort	Calcul du prix en direct, vérification d'adresse, aide à la saisie du mot de passe, version mobile des graphiques
Won't hors périmètre	Paiement en ligne, application mobile native, calcul d'itinéraire routier

3. Tableau Kanban

Le suivi des tâches se fait dans un tableau **Trello** organisé en colonnes :

Colonne	Contenu
Backlog	Toutes les user stories issues du cahier des charges, priorisées
À faire	Les cartes du lot en cours, prêtes à être développées
En cours	Une seule fonctionnalité à la fois (branche `feature/...` ouverte)
En test	Développement terminé, scénarios de test et vérifications visuelles en cours
Terminé	Testé et fusionné dans `develop`

Chaque carte contient la user story, ses critères d'acceptation (repris du cahier des charges), une étiquette de priorité (MoSCoW) et une étiquette de domaine (Front, Back, BDD, Sécurité, Déploiement, Documentation).

> **Lien du tableau : _____**

Exemples de user stories

En tant que…	je veux…	afin de…
visiteur	filtrer les menus par prix, thème, régime et nombre de personnes sans recharger la page	trouver rapidement un menu adapté
visiteur	voir les conditions d'un menu bien en évidence	ne pas être surpris par un délai ou une contrainte
utilisateur	voir le détail du prix (menu, remise, livraison) avant de valider	commander en connaissance de cause
utilisateur	modifier ou annuler ma commande tant qu'elle n'est pas acceptée	corriger une erreur sans appeler le traiteur
employé	faire avancer le statut d'une commande et prévenir le client automatiquement	suivre la production et la livraison
employé	indiquer le mode de contact et le motif avant d'annuler une commande	garder une trace de l'échange avec le client
administrateur	comparer le nombre de commandes et le chiffre d'affaires par menu sur une période	piloter l'offre de menus
administrateur	désactiver le compte d'un employé qui quitte l'entreprise	protéger l'accès à l'application

4. Planification et déroulement

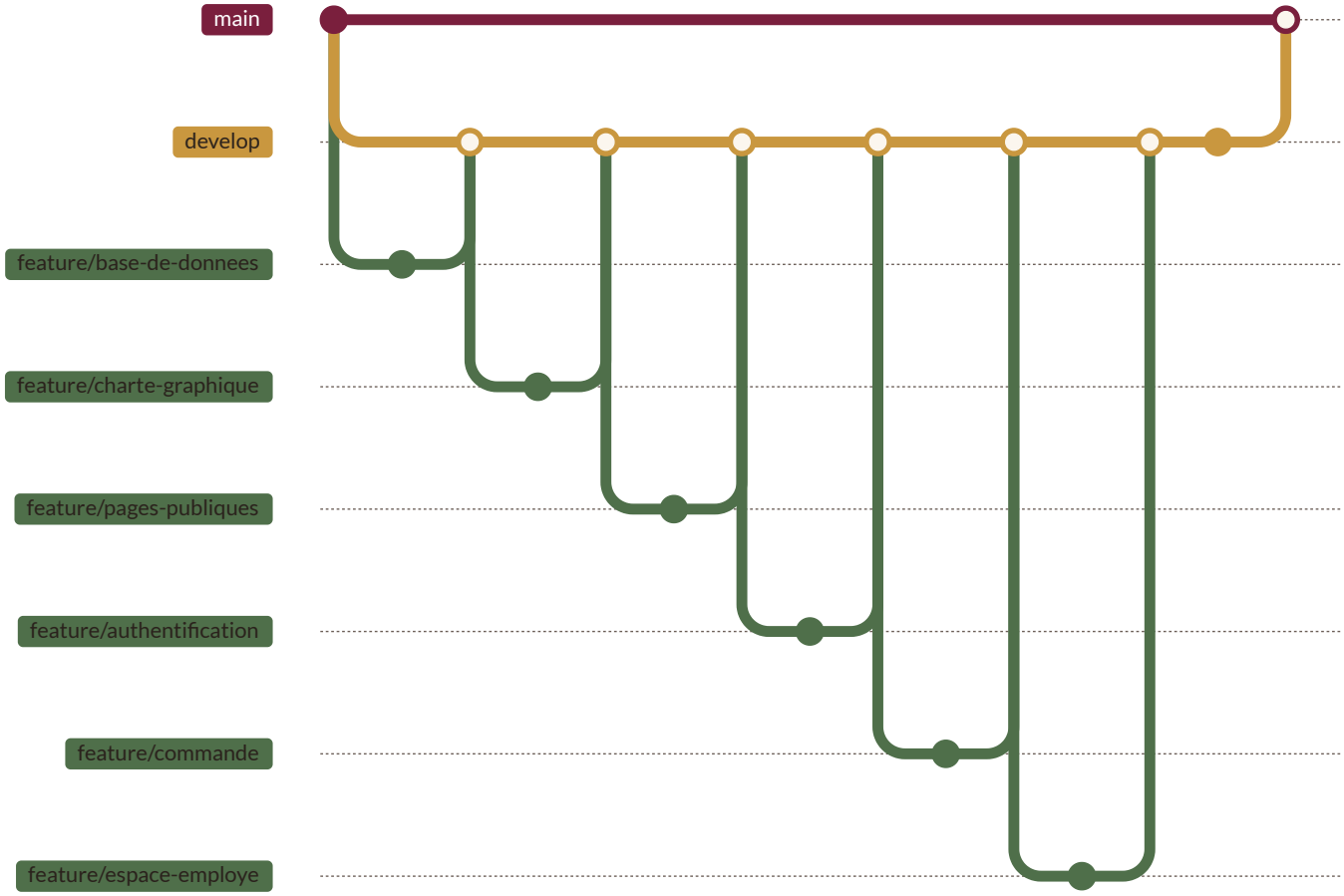
Le projet a été découpé en neuf lots successifs. L'ordre suit les dépendances techniques : la base de données et la charte d'abord, puis les fonctionnalités du visiteur vers l'administrateur, enfin la mise en production et la documentation.

#	Lot	Contenu	Branche Git
1	Initialisation	Analyse du sujet, choix de la stack, installation de l'environnement, dépôt GitHub, branches `main` / `develop`	`main`
2	Base de données	MCD complété, scripts de création et de données, collection MongoDB	`feature/base-de-donnees`
3	Charte et maquettes	Palette vérifiée (contrastes), typographie, 3 wireframes et 3 maquettes bureau et mobile	`feature/charte-graphique`
4	Pages publiques	Architecture MVC, accueil, catalogue filtrable, détail, contact, pages légales	`feature/pages-publiques`
5	Authentification	Inscription, connexion, rôles, mot de passe oublié, protection force brute	`feature/authentification`
6	Commande et espace client	Tunnel de commande, calcul du prix, suivi, modification, annulation, avis, profil	`feature/commande`
7	Espace employé	Gestion des commandes, menus, plats, horaires, avis, envoi d'images	`feature/espace-employe`
8	Espace administrateur	Comptes employés, statistiques MongoDB et graphiques	`feature/espace-admin`
9	Déploiement et documentation	Docker, Render, Aiven, Atlas, import des données, documents PDF	`feature/deploiement`, `feature/import-production`

5. Gestion du code source (Git)

5.1 Stratégie de branches

- `main` : branche principale, toujours stable ; c'est elle qui est déployée automatiquement en production.
- `develop` : branche d'intégration des fonctionnalités terminées et testées.
- `feature/...` : une branche par fonctionnalité, créée depuis `develop`. Après les tests, elle est fusionnée dans `develop` avec `--no-ff` (un commit de fusion garde la trace de la fonctionnalité dans l'historique).
- Quand `develop` est correctement testée, elle est fusionnée dans `main`, ce qui déclenche le déploiement.



Extrait de l'historique : les fonctionnalités suivantes (feature/espace-admin, feature/deploiement, feature/import-production) suivent exactement le même cycle.

5.2 Bonnes pratiques appliquées

- **Messages de commit explicites** en français : un titre court décrivant la fonctionnalité, puis la liste des changements.
- **Secrets jamais versionnés** : `.env` et les accès de production sont exclus par `.gitignore` ; seul `.env.example` (sans valeurs sensibles) est dans le dépôt.
- **Fins de ligne normalisées** (`.gitattributes`) pour un dépôt identique sous Windows et Linux.
- **Dépendances reproductibles** : `composer.lock` versionné, dossier `vendor/` exclu.
- **Déploiement continu** : toute fusion dans `main` est déployée automatiquement par Render.

6. Qualité, outils et difficultés rencontrées

6.1 Assurance qualité

- **Tests par scénarios** à chaque fonctionnalité : cas nominaux, cas d'erreur et tentatives d'attaque (injection, CSRF, accès à la commande d'un autre client, élévation de privilèges, faux fichier image…), via de vraies requêtes HTTP.
- **Vérifications visuelles** systématiques (captures d'écran bureau et mobile) comparées aux maquettes.
- **Contrôle de cohérence des données** : statistiques MongoDB comparées à un calcul direct en SQL.
- **Vérification syntaxique** de tous les fichiers PHP (`php -l`) avant chaque commit.

6.2 Outils

Besoin	Outil
Gestion des tâches	Trello (Kanban)
Versionnement	Git, GitHub, GitHub CLI
Développement	Visual Studio Code, XAMPP (PHP, MariaDB), MongoDB + mongosh, Composer
Maquettes et documents	HTML / CSS exportés en PDF, diagrammes Mermaid
Hébergement	Docker, Render, Aiven, MongoDB Atlas

6.3 Difficultés rencontrées et solutions

Difficulté	Solution apportée
La politique de sécurité (CSP) interdit les styles en ligne, nécessaires à un graphique classique.	Graphiques dessinés en SVG côté serveur (les dimensions sont des attributs, pas des styles), sans bibliothèque externe.
Derrière le proxy de l'hébergeur, tous les visiteurs semblaient avoir la même adresse IP (risque de bloquer tout le monde après des erreurs de connexion).	Lecture de l'IP réelle dans l'en-tête `X-Forwarded-For` quand `TRUST_PROXY` est activé ; la limite par e-mail reste la protection principale.
Le client MySQL de XAMPP ne gère pas l'authentification de MySQL 8 utilisée par l'hébergeur.	Script d'import en PHP (`database/import.php`) utilisant PDO, compatible et chiffré.
Une règle métier écrite en texte libre (« à commander 14 jours avant ») n'était pas vérifiable.	Ajout d'un champ « délai de commande en jours » au menu, contrôlé à la commande.
Risque de survente si deux clients commandent le dernier menu en même temps.	Transaction SQL avec verrou de ligne (`SELECT … FOR UPDATE`) sur le stock.
Version de PHP différente entre le poste (8.0) et la production (8.2).	Code limité aux fonctionnalités de PHP 8.0 ; image Docker pour figer l'environnement de production.
Peu de commandes dans les données de démonstration, graphiques peu parlants.	Ajout de commandes historiques aux prix calculés selon les règles réelles, et script de synchronisation MySQL → MongoDB.